Backend Engineer Technical Assignment V2

Interview Assignment

Introduction

For the interview we want you to complete a small assignment. You may choose one of two options.

Option 1

Complete the assignment at home and send us a link to your code repository. Alternatively, you can email us a zip file containing the code. Your commit history should show the steps you took to complete the assignment. Include a README.md file with instructions for running the service. We will review and run your code before the interview.

Option 2

Read the assignment, understand the requirements, and come to the interview prepared to write part of the code live. As you write the code, we will discuss and ask questions about your design decisions.

Option 1 is preferred, because it is more realistic. We understand that sometimes the circumstances do not allow you to complete the assignment at home, so we offer you option 2 as an alternative.

Mandatory Technologies

- Java 21 or newer
- Use either Spring Boot 3 or newer, or Spring Framework 6 or newer.
- Maven or Gradle as build tool
- Docker and Docker Compose for running the service
- Messaging system of your choice. Kafka is preferred.
- Relational database of your choice. PostgreSQL and MySQL are preferred.

Service Requirements

Write a small, self-contained service using the technologies listed above. Read everything carefully before you begin coding. The messaging system and database should run in Docker containers. The application can run locally or in a Docker container.

1. Expose a REST API with the following endpoint: `POST /user-posts/gather`

2. Read users from the following endpoint asynchronously: https://jsonplaceholder.typicode.com/users.
3. Read posts from the following endpoint asynchronously: https://jsonplaceholder.typicode.com/posts.
4. Filter all posts with titles longer than 200 characters.
5. Send the data in a messaging system.
6. Consume the data from the messaging system and save it in the database.
7. Expose the following REST API endpoint:
   - `GET /user-posts?page={page}&size={size}`: Get paginated user-posts data from the database, where `{page}` is the page number and `{size}` is the number of items per page.
   - The endpoint should return merged data of posts and users. Each post should contain user information within it. (postId, userName, userEmail, postTitle, postBody)
8. Write a test for the logic that merges posts and users.

Additional Details

Merging the data means taking the posts and adding user information to each post, such as userName and userEmail. Gathering users and posts should be done in parallel using asynchronous calls.